Forecasting Perishable-Product Demand With Gradient Descent

P1 Submission: Topic Background, Proposal, and Initial Data Exploration

NgocMaiNhi Nguyen
MSBA 265: Special Analytics Topics, University of the Pacific
Project: Grupo Bimbo Inventory Demand | October 2026

Part 1. Topic Background and Real-World Examples

Topic: demand forecasting for perishable products. Retailers and distributors must decide, before the week starts, how many units of every product to deliver to every store. For fresh bread, a forecast that is too low means empty shelves and lost sales, and one that is too high means returns and waste. Food waste is a large global problem: the United Nations Environment Programme [UNEP] (2021) estimates that about 931 million tonnes of food, roughly 17% of the food available to consumers, were wasted in 2019, and a share of it arises in retail and food services. Better demand forecasts are one practical lever for reducing it.

Examples of Demand Forecasting in Practice

Table 1
Real-World Examples of Demand Forecasting and Their Link to This Project

Setting	What is forecast	Why it matters for this project	Source
Retail inventory (Amazon research)	Probabilistic demand for many related products, learned from the history of all of them together	Shows that having the right stock in the right place and time is the business purpose of forecasting, and that models learn from the history of many items	Salinas et al. (2020)
Walmart (M5 forecasting competition)	Unit sales of thousands of products across stores in three U.S. states	A public, widely studied benchmark showing which approaches work on store-product sales data	Makridakis et al. (2022)
Grupo Bimbo (Kaggle competition)	Weekly demand per store, product, depot, and route for a bakery distributor in Mexico	This is the dataset and the business setting used in this project	Kaggle (2016a)
Retail forecasting practice and research	Replenishment and inventory decisions in retail	Reviews what works in practice and why simple, transparent benchmarks matter	Fildes et al. (2022)

How the Topic Connects to This Project

- The Grupo Bimbo competition (Kaggle, 2016a) asks for next-week demand per store and product, which is exactly the decision a distribution planner makes.
- Demand is strongly right-skewed (many small orders and a few very large ones), so the target is log-transformed before modeling (Hyndman & Athanasopoulos, 2021; Solis, 2026c).

- Gradient Descent (Bottou, 2010; Bottou et al., 2018; Robbins & Monro, 1951; Ruder, 2016) is used to learn a transparent linear model, so every forecast can be explained to a planner. A tree-based model such as XGBoost (Chen & Guestrin, 2016) is the natural comparison for later work.
- The winning Kaggle team relied on lagged demand and group averages as features (Kaggle, 2016b); this project uses the same idea in a simpler form, following the MSBA 265 workflow of exploratory data analysis, per-feature skewness checks, a multicollinearity audit, and linear regression (Solis, 2026c, 2026g).

FORECASTING BAKERY DEMAND

Teaching a model to predict how much bread each store needs

Case study: Grupo Bimbo Inventory Demand (weekly bakery deliveries to stores in Mexico)

THE PROBLEM: BAKERY PRODUCTS GO STALE FAST

TOO LITTLE
The shelf runs empty
= lost sales and unhappy customers

TOO MUCH
Bread is returned or thrown away
= waste and extra cost

About 931 million tonnes of food were wasted worldwide in 2019 (United Nations Environment Programme [UNEP], 2021).

13.7M	7	10	22 → 4	0.69
training rows used (no sampling)	weeks of data (weeks 3-9)	best-selling products	candidate → final features	R^2 on a week the model never saw

HOW IT WORKS: 5 STEPS

1 Clean — SQL renames columns and checks bad values →

2 Filter — SQL keeps the 10 top products in their own tables →

3 Remember — Python builds summaries of past sales per store and product →

4 Select — Keep only the features that add accuracy →

5 Learn — Gradient Descent tunes the formula

GRADIENT DESCENT AND THE RESULT

Walk downhill to less error

The model starts with a rough formula, measures its error, then moves each weight a small step in the direction that lowers the error. It repeats this until the error stops falling.

new weight = old weight − learning rate × slope of the error

The learning rate is the size of each step. Because the formula is a weighted sum, a planner can read every term.

Result on unseen week 9

$R^2 = 0.69$



R^2 = share of the ups and downs in demand that the formula explains (0 = none, 1 = all).

Under-forecast: 25.7% of demand (stockout risk)
Over-forecast: 14.1% of demand (excess-stock risk)

FROM FORECAST TO ACTION

Supply planners
Set the delivery quantity for each store and product

Depot and route managers
Fewer emergency re-deliveries and returns

Finance
Lower write-offs from stale, unsold bread

Limits: linear model; weeks 3-9 only; test labels are hidden, so R^2 is measured on week 9 of the training table. *Sources:* Kaggle (2016a); UNEP (2021); Salinas et al. (2020); Bottou et al. (2018); Solis (2026c). Full references are on the last pages.

Part 2. Project Proposal

2.1 Business Problem and Stakeholders

Organization (realistic scenario): the distribution planning team of a bakery company that delivers fresh products to stores every week. **Problem:** deliveries are planned with limited information about each store's future demand, so stores either run out of products or return unsold bread.

Decision: how many units of each product to load for each store next week.

Table 2
Stakeholders and the Decisions the Forecast Supports

Stakeholder	What they care about	Decision the forecast supports
Supply-chain and demand planners (primary users)	Right quantity, low waste	Weekly delivery quantity per store and product
Depot and route managers	Efficient routes, few emergency re-deliveries	Loading plan for each route
Store owners	Full shelves, few unsold items	Accepting or adjusting the proposed delivery
Finance	Cost of returns and write-offs	Budgeting waste and measuring savings

2.2 Project Question and Success Measure

Focused question. How accurately can the next-week and two-weeks-ahead demand of the best-selling products be forecast for every store using only past sales, with a linear regression trained by Gradient Descent, and how many units of stockout and excess stock would the remaining error represent?

Success measure. Primary: R^2 and RMSLE (root mean squared logarithmic error) on a validation week that the model never learned (week 9), compared with a baseline that always predicts the average. Proposed target: RMSLE clearly below the baseline and R^2 of at least 0.5, to be reviewed as results develop. Secondary: the share of demand that is under-forecast (stockout risk) and over-forecast (excess-stock risk).

2.3 Intended Use and What Would Make It Useful

The forecast is used as the default delivery quantity for each store-product, which planners can override. For a product that sells slowly and builds up unsold stock, the forecast lowers next week's quantity; for a product that sells well, it raises it. It is useful if it is more accurate than the simple average, can be explained (every term of the formula has a business meaning), and can be re-trained every week on a normal computer.

2.4 Dataset and Access

Source. The data come from the Kaggle competition *Grupo Bimbo Inventory Demand* (Kaggle, 2016a): a training file with weeks 3 to 9 and the target, and a test file with weeks 10 and 11 without the target. A free Kaggle account is required and the competition rules must be accepted before downloading.

Access and preparation plan. The raw training file has more than 70 million rows (74,180,464 rows once loaded) and the test file has 1,048,575 rows. The work is therefore split into two stages.

- **Stage 1: SQL Server.** Both files are loaded into SQL Server. SQL is used to clean the data: Spanish column names are renamed to English, a column-name typo is fixed, and every column is checked for NULL and invalid values (scripts in the repository folder *sql/*). *This applies the data-cleaning knowledge gained in the course MSBA 230, Database Management Systems with SQL and R*.
- **Stage 2: SQL filter.** SQL is then used to select the 10 best-selling products (largest total adjusted demand units in weeks 3-8) that were sold in every training week and appear in both test weeks, with at least 1,000 training rows per week and 100 test rows per week, and to export them into two separate SQL tables, *train_filtered* and *test_filtered*. The query design (grouping, joins, filtering, and creating tables from a query) also applies what I learned in that SQL course.
- **Stage 3: Python.** Only after these two tables exist does Python connect to SQL Server and read them week by week, in chunks of at most one million rows, to build the model. The script *01_test_connection.py* checks the connection between Python (VS Code) and SQL Server and counts the rows of both tables; its output (connection OK, 13,721,629 training rows and 169,578 test rows) confirms that the pipeline is connected.

Why only 10 products. With more than 70 million rows, the full table is big data for a single laptop and risks running out of memory or taking too long. Restricting the project to the 10 best-selling products reduces that risk while keeping the products that matter most for the business: the 10 products still contribute about 13.7 million training rows, and all of them are used (no sampling). A small synthetic database generator lets a reviewer run the whole pipeline without the Kaggle data.

What one row represents: the delivery of one product to one customer (store) in one week, through one depot, route, and sales channel.

Table 3
Columns of the Source Tables

Column (original name)	Table	Meaning
Week	train, test	Week number. Train: weeks 3-9. Test: weeks 10-11.
Agency_ID (Agencia_ID)	train, test	Sales depot that serves the stores.

Column (original name)	Table	Meaning
Channel_ID (Canal_ID)	train, test	Sales channel (for example, supermarket or small shop).
Route_SAK (Ruta_SAK)	train, test	Delivery route (several routes per depot).
Customer_ID (Cliente_ID)	train, test	Store or client identifier.
Product_ID (Producto_ID)	train, test	Product identifier.
Sales_Units_Today (Venta_uni_hoy)	train	Units delivered or sold that week.
Sales_Today (Venta_hoy)	train	Value of those sales in Mexican pesos.
Returns_Units_Next (Dev_uni_proxima)	train	Units returned the following week.
Returns_Next (Dev_proxima)	train	Value of the returned units in pesos.
Adjusted_Demand_Units (Demanda_uni_equil)	train	**Target.** Adjusted demand = units sold minus units returned (competition definition).
id	test	Row identifier used to submit predictions.

Data quality, privacy, and licensing limits. IDs are anonymized (no personal data); use is restricted to the competition's terms; the test table has no target, so a true test R^2 cannot be computed; the test table as loaded has only 1,048,575 rows (the Excel row limit), so it is probably a truncated copy; only seven training weeks exist, so seasonality cannot be learned; and there is no information on promotions, competitor prices, or weather.

2.5 Features Used by the Model

The test table contains only identifiers and the week, so the model cannot use the sales of the week being forecast (that would be data leakage). Instead, every feature is calculated **only from earlier weeks** of the same store, product, depot, route, or channel. Table 4 defines all 22 candidate features, what each one means, and how it is calculated. The text below the table explains the building blocks.

Table 4
Candidate Features: Meaning and Calculation

Feature	Type	What it means	How it is calculated
te_product	Average	Typical demand level of the product	(sum of log1p(demand) of the product's earlier rows + m*G) / (number of those rows + m)
te_customer	Average	Typical demand level of the store	(sum of log1p(demand) of the store's earlier rows + 5·G) / (number of those rows + 5)

Feature	Type	What it means	How it is calculated
te_agency	Average	Typical demand level of the depot	(sum of log1p(demand) of the depot's earlier rows + 5·G) / (number of those rows + 5)
te_route	Average	Typical demand level of the delivery route	(sum of log1p(demand) of the route's earlier rows + 5·G) / (number of those rows + 5)
te_channel	Average	Typical demand level of the sales channel	(sum of log1p(demand) of the channel's earlier rows + 5·G) / (number of those rows + 5)
te_cust_prod	Average	Typical demand of this store for this product	(sum of log1p(demand) of the earlier rows of the store-product pair + 5·G) / (number of those rows + 5); if the pair is new, the product's te_product is used
logcnt_product	Count-based	How often the product is ordered	log(1 + earlier order lines of the product / weeks of history)
logcnt_customer	Count-based	How often the store orders	log(1 + earlier order lines of the store / weeks of history)
logcnt_agency	Count-based	How busy the depot is	log(1 + earlier order lines of the depot / weeks of history)
logcnt_route	Count-based	How busy the route is	log(1 + earlier order lines of the route / weeks of history)
logcnt_channel	Count-based	How busy the sales channel is	log(1 + earlier order lines of the channel / weeks of history)
logcnt_cust_prod	Count-based	How regularly the store orders this product	log(1 + earlier order lines of the store-product pair / weeks of history)
last_cust_prod	Earlier order	Most recent demand of the store for this product	log1p(demand) in the latest earlier week in which the pair was ordered; if new, te_cust_prod
prev_cust_prod	Earlier order	The order before the most recent one	log1p(demand) of the second-latest earlier order of the pair; if missing, last_cust_prod
mean3_cust_prod	Earlier order	Average of the last three orders	mean of last_cust_prod, prev_cust_prod and the third-latest order of the pair
gap_cust_prod	Count-based	Weeks since the store last ordered this product	log(1 + (current week - week the pair was last seen)); 0 if never seen

Feature	Type	What it means	How it is calculated
ret_cust_prod	Rate	Share of this store's product that came back unsold	sqrt of (returned units + 5*p) / (sold units + 5), where p = the product's overall return rate; the square root reduces the right skew
log_hist_price	Volume	Price level of the product	log(1 + historical sales value in pesos / units sold) of the product
hist_return_rate	Rate	Share of the product that usually comes back unsold	historical returned units / units sold of the product (0 to 1)
is_new_customer	Flag	Store never seen before	1 if the store has no earlier history, otherwise 0
is_new_product	Flag	Product never seen before	1 if the product has no earlier history, otherwise 0
is_new_cust_prod	Flag	Store has never ordered this product before	1 if the store-product pair has no earlier history, otherwise 0

Note. All features use only weeks before the row's week. G is the overall average of log(1 + demand) in the history and $m = 5$ is a smoothing weight that pulls small groups toward G. "Pair" means one store-product combination.

Building blocks.

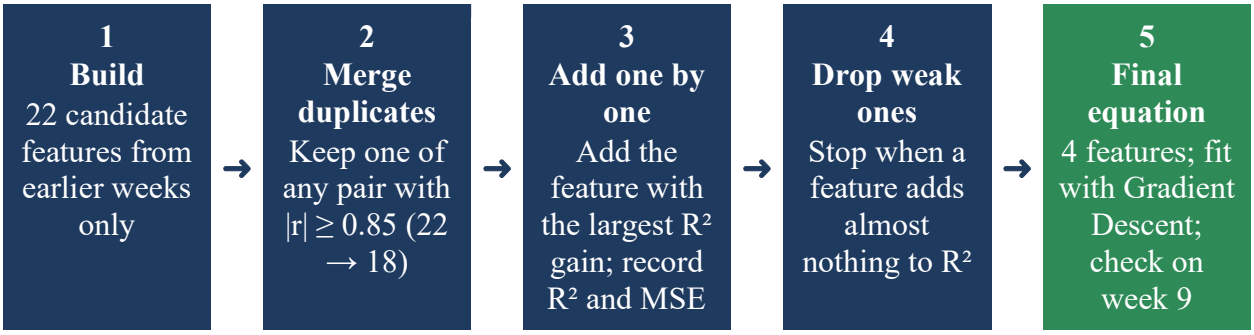
- **Average features** (names starting with *te_*) apply the target-encoding idea taught in the course (Solis, 2026d; Micci-Barreca, 2001): instead of turning thousands of Customer_ID or Product_ID values into thousands of columns, each ID is replaced by one number, the average demand seen for that ID in earlier weeks.
- **Count-based features** (names starting with *logcnt_*) describe how often an ID appears. Because they are counts that grow with the number of weeks, they are divided by the number of weeks of history and put on a log scale; they are not transformed a second time.
- **Earlier-order features** (*last_*, *prev_*, *mean3_*) are the lagged demand of the same store and product, the ingredient that the winning Kaggle team found most useful (Kaggle, 2016b).
- **Rates and flags** describe the share of product returned unsold and whether a store, product, or pair is new. For a new item there is no history, so it receives the overall average and a flag set to 1.

Illustration (hypothetical numbers). If a store's latest earlier order of a product was 7 units, then *last_cust_prod* = log(1 + 7) = 2.08. If the same store has ordered the product in 4 of the last 8 weeks, *logcnt_cust_prod* = log(1 + 4/8) = 0.41.

From 22 candidate features to the final equation. Instead of using every feature, the project follows the five-step workflow in Figure 1 and keeps only the features that improve the fit (see the selection idea below).

Figure 1

Feature-Selection Workflow From 22 Candidates to the Final Equation



Note. R^2 = share of the variation in log(1 + demand) explained; MSE = mean squared error.

Selection idea (best-fit model, as in JMP). In the course MSBA 240, Advanced Business Analytics, I learned how JMP builds a best-fit model: it keeps only the terms that contribute to the fit rather than entering every variable. This project applies the same idea: start from the intercept alone, add the feature that improves R^2 the most, record R^2 and MSE at every step, and stop adding features once the improvement becomes negligible. The extra features would complicate the equation without making the forecast better.

In the full run on the 10 best-selling products this left **4 features**, which form the final equation: *mean3_cust_prod*, *te_product*, *te_customer* and *is_new_cust_prod*. With only these four, the training R^2 (exact least squares) is 0.660, compared with 0.667 when all 18 non-duplicate candidates are used, so the other 14 features add almost nothing.

2.6 Planned Method and Baseline

Model. The forecast is a multiple linear regression, $\hat{y} = w0 + w1 \cdot x1 + w2 \cdot x2 + \ldots + wk \cdot xk$, where $\hat{y}$ is the forecast of log(1 + demand) and x1 … xk are the features of Table 4. The same method was first applied to build models with one and with two features (Nguyen, 2026); this project scales it up to a table with millions of rows and many candidate features. It also extends the single-feature model of the course lab (expenses = w0 + w1 × BMI) to several features, exactly as the Project 1 assignment does with two features (Solis, 2026f). The intercept w0 is the baseline forecast when every feature is zero, and each weight is the expected change in the forecast for a one-unit change in its feature (Solis, 2026h).

Two ways to fit the same model. The course presents two strategies for finding the weights (Solis, 2026e). The *Normal Equation*, θ = (X'X)^-1 X'y, solves for the weights directly, with no learning rate and no epochs. *Gradient Descent* learns them step by step: initialize the weights, predict, measure the error, summarize it as the loss (mean squared error, MSE), compute the gradients, update the weights, and repeat until the loss flattens (Solis, 2026e).

Model used in this project: multiple linear regression trained with mini-batch Gradient Descent. Gradient Descent is the main method because it works with millions of rows and many features, and its mini-batch version learns from one chunk of data at a time (Bottou, 2010; Bottou et al., 2018).

The Normal Equation is used as a cross-check: if Gradient Descent has converged, both should give nearly the same weights, as in the course comparison (Solis, 2026f). As in the course lab, features are standardized before Gradient Descent and the learned weights are converted back to the original units, so the final equation can be read directly (Solis, 2026f, 2026h).

Parameters and hyperparameters. The weights are *parameters*: they are learned from the training data. The learning rate, the number of epochs per chunk, and the mini-batch size are *hyperparameters*: settings that we choose to control how the model learns (Solis, 2026h). The learning rate is chosen by watching the loss curve: it is lowered if the loss oscillates or diverges and raised if the loss falls too slowly (Solis, 2026e).

Training, validation, and test roles. The course rule is that training data teach the weights, validation data help choose settings, and test data give the final evaluation (Solis, 2026e), a loop also described as fitting on training data, choosing on validation data, and reporting on untouched test data (Solis, 2026a). Because the data are weekly, the split is chronological, which also avoids leakage from the future (Solis, 2026a). Table 5 shows how the weeks are used.

Table 5
Roles of the Data in the Modeling Plan

Role	Data	Used for	Weights updated from it?
Training	Weeks 4-8 (week 3 only supplies history for the features of week 4)	Learn the weights by Gradient Descent; choose which features to keep	Yes
Validation	Week 9, held out	Check that R^2 is close to the training R^2; compare with the baselines; freeze the final equation	No
Test	Weeks 10-11 (target hidden)	Apply the frozen equation once to produce the forecasts	No

Note. Because the test table has no target, a true test R^2 cannot be calculated; the validation R^2 is the estimate of how the equation will perform.

Baselines and metrics. Every result is compared with simple alternatives on the same validation rows (Table 6), following the course practice of comparing against a baseline model (Solis, 2026f). Results are reported with the metrics taught in the course: MSE, RMSE, MAE, and R^2 (Solis, 2026e, 2026f). RMSE on the log scale is the competition metric RMSLE. R^2 is read as the share of the variation in demand that the model explains, not as "accuracy" (Solis, 2026e).

Residuals (actual minus predicted) are also inspected to see whether the forecast is systematically too high or too low.

Table 6
Models Compared in the Modeling Plan

Model	What it is	Why it is included
Baseline 1: average	Always predicts the average log demand of the training weeks	The minimum any useful model must beat ($R^2 = 0$ by definition)
Baseline 2: best single feature	Linear regression with only the one feature that raises R^2 the most (step 1 of the feature ladder)	Mirrors the one-feature baseline of the course assignment; shows what additional features add (Solis, 2026f)
Normal Equation	Exact least-squares weights of the final features	Reference: the best possible linear model for these features
Gradient Descent (final model)	Mini-batch Gradient Descent on the final features	The proposed method; should match the Normal Equation if it has converged

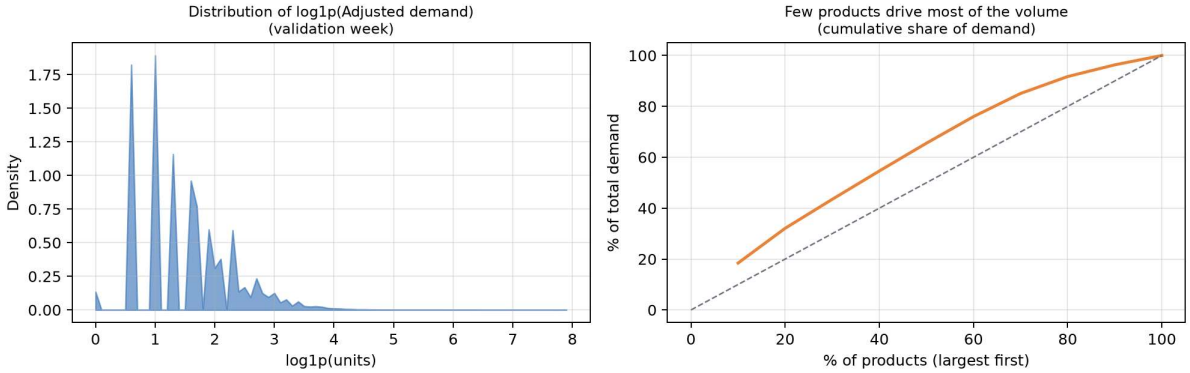
Same preparation at forecast time. A model that works in training but fails on new records is a pipeline problem, so the preparation learned from the training weeks (the history statistics and the scaler) must be reproduced exactly when new weeks are forecast (Solis, 2026h). It is fitted only on training rows to avoid leakage (Solis, 2026a), saved together with the model, and reloaded in a demonstration that must reproduce the same predictions. The work is implemented in Python with NumPy (Harris et al., 2020), pandas (McKinney, 2010), scikit-learn for cross-checking (Pedregosa et al., 2011), and Matplotlib for charts (Hunter, 2007).

Part 3. Initial Data Exploration

Table 7
Data Profile of the Filtered Tables

Item	Value
Products in scope	10 (1240, 1242, 1250, 1278, 1284, 2233, 2425, 36610, 43285, 45143)
Training rows used (all rows, no sampling)	13.7 million (13,721,629)
Test rows (weeks 10-11)	169,578
Training weeks / validation week	3-8 (week 3 builds the history) / 9
Test weeks	10 and 11

Figure 2
Distribution of the Target (Log Scale) and Share of Demand by Product



Note. Left: histogram of log(1 + adjusted demand). Right: share of total demand explained by products, largest first.

Interpretation. Adjusted demand is strongly right-skewed (skewness 24.14); after the log(1 + x) transformation, skewness is 0.54, which is within the range the course treats as acceptable (|skewness| ≤ 1; Solis, 2026c). The median order is 5 units while the 99th percentile is 80 (maximum 3,850), so a few large orders must not dominate the fit. Same-week sales units are almost identical to demand (correlation 0.998) but do not exist for the test weeks, so they are not used as a feature.

Skewness, Transformation Decisions, and Multicollinearity

Following the course workflow (Solis, 2026c), skewness is measured **separately for every variable** with the univariate skewness coefficient: |skewness| > 1 is moderately skewed and |skewness| > 3 is extremely skewed. A strongly skewed variable is not transformed automatically. The decision depends on what the variable measures (Solis, 2026b): a continuous

positive volume may be log-transformed, but a count, a rate, or a 0/1 flag should not be transformed blindly. Table 8 summarizes the decision for each type of variable used here.

Table 8
Transformation Decision by Variable Type

Variable type	Variables	Decision
Target (continuous positive volume of units)	Adjusted_Demand_Units	log(1 + x). Skewness falls from 24.14 to 0.54. Zero demand means a fully returned delivery and is handled by the +1; the log scale also matches the competition metric RMSLE.
Average of log demand	te_ features	No transformation: already an average of log values.
Count-based	logcnt_ features, gap_cust_prod	No second transformation: already divided by weeks of history and put on a log scale.
Earlier order	last_, prev_, mean3_ features	No transformation: already log demand.
Rate (0 to 1)	hist_return_rate, ret_cust_prod	Bounded; a square root is used for ret_cust_prod, whose right tail is long.
Flag (0/1)	is_new_ features	No transformation.

Note. Skewness of every feature is computed on all training rows in train_full_filtered.ipynb, section 4.1, and saved with the feature definitions (outputs/final/feature_dictionary.csv).

The target is a count of units, and the course cautions against logging counts (Solis, 2026b). The log transformation is kept here for three reasons: units per delivery range widely (median 3, 99th percentile 41, so the variable behaves like a volume), the skewness is extreme, and the competition metric is itself defined on the log scale. A Poisson or negative binomial model is a possible alternative for later work (Solis, 2026c).

For **multicollinearity**, the course threshold is a Pearson correlation of $|r| \geq 0.85$ between two predictors (Solis, 2026c). Feature pairs above this threshold are merged by keeping the one more strongly related to demand. Extreme demand values above the 99.9th percentile are capped in the training rows only (never in validation or test) so that a few giant orders do not dominate the fit.

Missing Values, Unusual Patterns, and Preparation Decisions

- **Missing and invalid values:** checked in SQL and again in train_full_filtered.ipynb on all rows (NULLs, negative values, zeros).
- **Skewness:** log(1 + x) for the target; predictions are converted back with exp(x) − 1.
- **Leakage:** features use only earlier weeks; the first week only builds history.

- **New customers or products:** no history exists, so they receive the overall average and a flag; the share of such rows is reported for the test weeks.
- **Large data:** every week is read and processed in chunks of at most one million rows.

Part 4. Feasibility and Next Steps

A first working version exists, which shows that the approach is feasible. Table 9 shows results on validation week 9, which the model never learned.

Table 9
Feasibility Test on Validation Week 9

Model	R^2 (log scale)	RMSLE
Baseline: always the average	-0.001	0.917
Gradient Descent (linear model)	0.691	0.510
Exact least squares (reference)	0.685	0.514

Note. Gradient Descent reaches almost the same quality as the exact solution, so the algorithm converges. The target set in Part 2 ($R^2 \geq 0.5$) is met. Training R^2 is 0.656, close to the validation value, so the model is not overfitting. RMSLE of the baseline and of exact least squares is derived from the R^2 values; ranking products by units favors bulky, high-variance products, which limits how far a linear model can go.

Table 10 shows the feature ladder: the intercept, the coefficient of the feature that has just entered, and R^2 and MSE on the training weeks and on validation week 9 as each feature is added. One feature (the average of the last three orders of the same store and product) already explains most of the variation. After the fourth feature the curve is flat: the other 14 features together add less than 0.007 to R^2, so they are dropped.

Table 10
Feature Ladder: Intercept, Coefficient, R^2, and MSE as Each Feature Enters the Equation

Step	Feature added	Intercept	Coefficient	R^2 train	MSE train	R^2 valid.	MSE valid.
0	(intercept only)	1.9874	-	0.0000	0.8359	-0.0013	0.8408
1	mean3_cust_prod	0.2069	0.8794	0.6182	0.3192	0.6661	0.2804
2	te_product	-0.1021	0.2850	0.6341	0.3058	0.6693	0.2777
3	te_customer	-0.9669	0.4567	0.6497	0.2928	0.6812	0.2677
4	is_new_cust_prod	-0.8517	-0.2721	0.6598	0.2843	0.6851	0.2645
5-18	14 further features	-0.0511	-	0.6668	0.2785	0.6901	0.2603

Note. Rows 1-4 form the final equation (exact least squares). The last row shows the values after all 18 non-duplicate candidates have been added; the coefficient column is omitted there because it is listed for individual features only.

What remains unknown: how much accuracy a linear model loses on the two-weeks-ahead forecast; how many test customers have no history; and whether further history-based features can raise R^2 beyond about 0.69, because the ladder is already flat.

Next steps:

- Run the forecast for test weeks 10 and 11 and check the coverage of customers and products.
- Translate the forecast error into units of stockout and excess stock for each product.
- Add the data-to-action pipeline, the saved-model demonstration, a stakeholder presentation, and the README for the P2 package.
- Push the project to the cloud repository (https://github.com/luckynnmn/bimbo-demand-forecast-gradient-descent) and complete the Assignment Validation form.

References

Bottou, L. (2010). Large-scale machine learning with stochastic gradient descent. In Y. Lechevallier & G. Saporta (Eds.), *Proceedings of COMPSTAT 2010* (pp. 177-186). Springer. https://doi.org/10.1007/978-3-7908-2604-3_16

Bottou, L., Curtis, F. E., & Nocedal, J. (2018). Optimization methods for large-scale machine learning. *SIAM Review, 60*(2), 223-311. https://doi.org/10.1137/16M1080173

Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 785-794). ACM. https://doi.org/10.1145/2939672.2939785

Fildes, R., Ma, S., & Kolassa, S. (2022). Retail forecasting: Research and practice. *International Journal of Forecasting, 38*(4), 1283-1318. https://doi.org/10.1016/j.ijforecast.2019.06.004

Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M. H., Brett, M., Haldane, A., del Rio, J. F., Wiebe, M., Peterson, P., . . . Oliphant, T. E. (2020). Array programming with NumPy. *Nature, 585*(7825), 357-362. https://doi.org/10.1038/s41586-020-2649-2

Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The elements of statistical learning: Data mining, inference, and prediction* (2nd ed.). Springer. https://doi.org/10.1007/978-0-387-84858-7

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering, 9*(3), 90-95. https://doi.org/10.1109/MCSE.2007.55

Hyndman, R. J., & Athanasopoulos, G. (2021). *Forecasting: Principles and practice* (3rd ed.). OTexts. https://otexts.com/fpp3/

Kaggle. (2016a). *Grupo Bimbo inventory demand* [Data set]. https://www.kaggle.com/c/grupo-bimbo-inventory-demand

Kaggle. (2016b). *Grupo Bimbo inventory demand: 1st place solution summary* [Competition forum post]. https://www.kaggle.com/c/grupo-bimbo-inventory-demand/discussion

Makridakis, S., Spiliotis, E., & Assimakopoulos, V. (2022). M5 accuracy competition: Results, findings, and conclusions. *International Journal of Forecasting, 38*(4), 1346-1364. https://doi.org/10.1016/j.ijforecast.2021.11.013

McKinney, W. (2010). Data structures for statistical computing in Python. In S. van der Walt & J. Millman (Eds.), *Proceedings of the 9th Python in Science Conference* (pp. 56-61). https://doi.org/10.25080/Majora-92bf1922-00a

Micci-Barreca, D. (2001). A preprocessing scheme for high-cardinality categorical attributes in classification and prediction problems. *ACM SIGKDD Explorations Newsletter, 3*(1), 27-32. https://doi.org/10.1145/507533.507538

Nguyen, N. M. N. (2026). *Linear regression application: One and two features* [Source code]. GitHub. https://github.com/luckynnmn/linear-Regression-application-1-2-features

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825-2830. https://jmlr.org/papers/v12/pedregosa11a.html

Robbins, H., & Monro, S. (1951). A stochastic approximation method. *The Annals of Mathematical Statistics, 22*(3), 400-407. https://doi.org/10.1214/aoms/1177729586

Ruder, S. (2016). *An overview of gradient descent optimization algorithms*. arXiv. https://arxiv.org/abs/1609.04747

Salinas, D., Flunkert, V., Gasthaus, J., & Januschowski, T. (2020). DeepAR: Probabilistic forecasting with autoregressive recurrent networks. *International Journal of Forecasting, 36*(3), 1181-1191. https://doi.org/10.1016/j.ijforecast.2019.07.001

Solis, S. (2026a). *Classification foundations lecture* [Lecture slides]. In *MSBA 265: Special analytics topics, Project 2: Classification*. GitHub. https://github.com/shylasolis/machine-learning-pathway/tree/main/Project_02_Classification

Solis, S. (2026b). *Demystifying data: Counts, volumes, and the danger of automated tools* [Lesson handout]. In *MSBA 265: Special analytics topics, Module 1*. GitHub. https://github.com/shylasolis/machine-learning-pathway/tree/main/Module_01_EDA

Solis, S. (2026c). *Foundational module 1: Step-by-step hands-on lab guide* [Workshop handout]. In *MSBA 265: Special analytics topics, Module 1*. GitHub. https://github.com/shylasolis/machine-learning-pathway/tree/main/Module_01_EDA

Solis, S. (2026d). *Handling high-cardinality features* [Lesson handout]. In *MSBA 265: Special analytics topics, Module 1*. GitHub. https://github.com/shylasolis/machine-learning-pathway/tree/main/Module_01_EDA

Solis, S. (2026e). *Linear regression: Two training strategies* [Student handout]. In *MSBA 265: Special analytics topics, Module 2: Preprocessing, models, and diagnostics*. GitHub. https://github.com/shylasolis/machine-learning-pathway/tree/main/Module_02_Preprocessing_Models_Diagnostics

Solis, S. (2026f). *Linear regression lab: Scripts 01-04 and assignment* [Source code and assignment]. In *MSBA 265: Special analytics topics, Project 1: Linear regression*. GitHub. https://github.com/shylasolis/machine-learning-pathway/tree/main/Project_01_Linear_Regression

Solis, S. (2026g). *Machine-learning-pathway: Special analytics topics* [Course materials repository]. GitHub. https://github.com/shylasolis/machine-learning-pathway

Solis, S. (2026h). *Module 1 expanded study guide* [Study guide]. In *MSBA 265: Special analytics topics, Module 2: Preprocessing, models, and diagnostics*. GitHub. https://github.com/shylasolis/machine-learning-pathway/tree/main/Module_02_Preprocessing_Models_Diagnostics

United Nations Environment Programme. (2021). *Food waste index report 2021*. https://www.unep.org/resources/report/unep-food-waste-index-report-2021